



E-SPORTS 1GSY

Autor: Javier Jimenez Alonso

Curso 2025-2026 | 1º DAW | Program. Tutor: Javier Seijas Bellido

1. Descripción general del proyecto

Este proyecto consiste en el desarrollo de una aplicación de consola en Java que permite gestionar de forma integral una liga de e-sports. El sistema está orientado a la empresa organizadora 1GSY y cubre todo el ciclo de vida de una competición: registro de personas, gestión de equipos y plantillas, generación de calendario, disputa de partidos, control de incidencias y sanciones, y obtención de clasificaciones y estadísticas finales.

La aplicación está estructurada en capas bien diferenciadas: una capa de interfaz de usuario (Main con menús interactivos), una capa de lógica de gestión (Liga) y una capa de entidades de dominio (Jugador, Entrenador, Equipo, Partido, IncidenciaLog). Esta separación garantiza que cada componente tenga una única responsabilidad, facilitando el mantenimiento y la extensión futura del sistema.

El proyecto integra de forma coherente todos los contenidos trabajados durante el curso: herencia, clases abstractas, interfaces, arrays estáticos, listas dinámicas, matrices, colas FIFO, pilas LIFO, tablas hash, excepciones personalizadas y polimorfismo.

1.1. Contexto del dominio

El juego base de la competición es SMITE, un MOBA en el que cada equipo necesita exactamente cinco jugadores en roles concretos: TOP, JUNGLE, MID, ADC y SUPPORT. Esta restricción tiene un impacto directo en el diseño de datos: el array de titulares tiene exactamente cinco posiciones y el sistema impide que dos titulares compartan el mismo rol dentro de un equipo.

La temporada se organiza en 14 jornadas con hasta 5 partidos por jornada. Los partidos se generan automáticamente como un calendario de liga y se gestionan mediante una cola FIFO que asegura el orden cronológico de disputa.

1.2. Funcionalidades principales

- Gestión completa de personas: alta, baja, búsqueda y modificación de jugadores y entrenadores.
- Gestión de equipos: creación, asignación de entrenador, fichaje de titulares y suplentes, control de presupuesto.
- Calendario de jornadas: generación automática, visualización completa y consulta por jornada.
- Cola de partidos: gestión FIFO con disputa ordenada, validación de convocatorias y registro de resultados.
- Incidencias y sanciones: registro, consulta por equipo o jugador, bloqueo de convocatoria para sancionados.
- Historial de acciones: pila LIFO con registro automático, consulta de última acción y funcionalidad de deshacer.
- Estadísticas y clasificación: ordenadas automáticamente por victorias, diferencia de puntos y nombre de equipo.
- Control de errores: seis excepciones personalizadas que protegen todas las reglas de negocio críticas.

2. Diseño de clases

El sistema está formado por once clases y una interfaz, organizadas en cuatro paquetes. El diagrama siguiente muestra las relaciones principales:

Clase / Interfaz	Tipo	Responsabilidad principal
PersonaLiga	Clase abstracta	Agrupar atributos y comportamiento comunes a jugadores y entrenadores
Jugador	Clase concreta	Extiende PersonaLiga; añade rol, niveles, MVPs y estado de sanción
Entrenador	Clase concreta	Extiende PersonaLiga; añade experiencia, especialidad y victorias
Entrenable	Interfaz	Define el contrato de entrenamiento y cálculo de rendimiento
Equipo	Clase concreta	Gestiona plantilla (array titulares + lista suplentes) y presupuesto
Partido	Clase concreta	Representa un enfrentamiento; encapsula registro de resultados
IncidenciaLog	Clase concreta	Registro de incidencias con referencias nullable a entidades afectadas
PersonajeJuego	Clase concreta	Personaje de SMITE usado por un jugador; requiere equals/hashCode
EstadisticasDelPickPersonaje	Clase auxiliar	Acumula partidas y victorias por personaje dentro del HashMap
Liga	Clase gestora	Gestiona todas las estructuras globales y coordina las operaciones
Main	Clase de entrada	Menús de consola; llama a Liga y muestra resultados al usuario

2.1. Jerarquía de herencia

La clase abstracta `PersonaLiga` actúa como raíz de la jerarquía de personas. Declara `calcularCosteMensual()` como método abstracto, obligando a cada subclase a implementar su propia fórmula económica. Contiene atributos comunes (identificador, nombre, nickname, edad, `salarioBase`)

`Jugador` implementa además la interfaz `Entrenable` y `Comparable<Jugador>`. `Entrenador` no implementa `Entrenable` porque un coach no mejora sus habilidades del mismo modo que un jugador.

`Liga` mantiene una lista única `ArrayList<PersonaLiga>` para ambos tipos, usando `instanceof`

2.2. Estructuras de datos utilizadas

Estructura	Ubicación	Propósito
Jugador[] titulares (tamaño 5)	Equipo	Array fijo que garantiza exactamente 5 titulares por equipo
ArrayList<Jugador> suplentes	Equipo	Lista dinámica de suplentes sin límite fijo
ArrayList<PersonaLiga> personas	Liga	Lista principal de todas las personas registradas
ArrayList<Equipo> equipos	Liga	Lista de todos los equipos de la liga
ArrayList<Partido> partidosJugados	Liga	Historial de partidos ya disputados
ArrayList<IncidenciaLog> incidencias	Liga	Registro de todas las incidencias
HashSet<String> idsPersonas	Liga	Evita duplicación de IDs de personas
HashSet<String> nombresEquipos	Liga	Evita duplicación de IDs de equipos
HashSet<String> idsPartidos	Liga	Evita duplicación de IDs de partidos
Queue<Partido> colaPartidosPendientes	Liga	Cola FIFO para procesar partidos en orden cronológico
Stack<String> pilaHistorialAcciones	Liga	Pila LIFO con historial de operaciones y deshacer
Partido[][] calendario (14x5)	Liga	Matriz para consulta directa por jornada
Map<PersonajeJuego, Estadisticas>	Jugador	HashMap para estadísticas por personaje

3. Architecture Decision Records (ADRs)

Cada decisión de diseño relevante del proyecto está documentada como un ADR (Un saludo a Pablo por enseñarme qué era un ADR). Este formato describe el contexto del problema, la decisión adoptada y la justificación técnica de por qué se eligió esa solución frente a las alternativas.

ADR-001 — Jerarquía de herencia con clase abstracta `PersonaLiga`

Contexto

La liga gestiona jugadores y entrenadores. Ambos comparten datos básicos y operaciones comunes, pero difieren en atributos específicos y en cómo calculan su coste mensual. Modelarlos como clases independientes duplicaría todos los atributos comunes y obligaría a mantener dos listas separadas en Liga.

Decisión

Se define `PersonaLiga` como clase abstract con los atributos comunes (identificador, nombre, nickname, edad, salarioBase) y el método abstracto `calcularCosteMensual()`. `Jugador` y `Entrenador` la extienden añadiendo sus propios atributos. Liga almacena ambos tipos en una única `ArrayList<PersonaLiga>`.

Justificación

El polimorfismo permite tratar jugadores y entrenadores de forma uniforme donde sea posible, usando `instanceof` con pattern matching solo cuando se necesitan atributos específicos. Declarar `calcularCosteMensual()` abstracto garantiza que ninguna subclase futura pueda olvidar implementarla.

ADR-002 — Array fijo de 5 posiciones para jugadores titulares

Contexto

Cada equipo tiene exactamente 5 jugadores titulares, uno por rol. Este número no cambia: las reglas del juego no permiten ni más ni menos. Usar un `ArrayList` habría permitido añadir un sexto titular sin ninguna validación estructural.

Decisión

Se usa `Jugador[] titulares = new Jugador[5]` en `Equipo`. El tamaño es invariable por definición del lenguaje. Las posiciones vacías son null hasta que se ficha el jugador correspondiente. `ficharTitular()` recorre el array comprobando rol duplicado antes de insertar.

Justificación

El propio tipo de dato impone la restricción sin necesidad de lógica adicional. El array contrasta intencionadamente con `ArrayList<Jugador>` suplentes, que es dinámico porque el número de suplentes no tiene límite fijo en las reglas de la competición.

ADR-003 — HashSet auxiliar para deduplicación en tiempo constante

Contexto

La liga no puede tener dos personas con el mismo ID, dos equipos con el mismo nombre ni dos partidos con el mismo ID.

Decisión

Se mantienen tres `HashSet<String>` auxiliares en Liga: `idsPersonasRegistradas`, `nombresEquipos` e `idsPartidos`. Antes de cada inserción se comprueba `set.contains(clave)`. Si devuelve `true` se lanza la excepción correspondiente, si no, se añade simultáneamente a la lista principal y al set para mantenerlos sincronizados.

Justificación

Cuando se elimina una persona, su ID también se elimina del set para liberar el identificador.

ADR-004 — Queue FIFO para partidos pendientes de disputar

Contexto

Los partidos deben disputarse en el orden en que fueron programados. Ninguno puede saltarse la cola.

Decisión

Se declara `colaPartidosPendientes` como `Queue<Partido>` implementada con `LinkedList`. Los partidos se encolan con `offer()` durante la generación del calendario. La disputa usa `peek()` para consultar sin extraer y `poll()` para extraer solo cuando todo el proceso es correcto. `mostrarPartidosPendientes()` itera sin modificar la cola.

Justificación

Si en algún momento se dispara una convocatoria no válida, el partido no desaparece, puede mantenerse hasta que se resuelva la incidencia, por lo que es realmente útil usar las colas y mantiene el orden de jornadas

ADR-005 — Stack LIFO para historial de acciones

Contexto

El sistema necesita registrar todas las operaciones importantes y permitir ver la última acción, ver el historial completo y deshacer la operación más reciente. La acción más reciente es siempre la más relevante.

Decisión

Se declara `pilaHistorialAcciones` como `Stack<String>` en Liga. Cada operación relevante llama a `registrarAccion()` que hace `push()`. `mostrarUltimaAccion()` usa `peek()` sin modificar la pila. `mostrarHistorial()` crea una copia temporal para recorrerla con `pop()`. `deshacerUltimaAccion()` extrae con `pop()` el elemento del tope.

Justificación

El deshacer elimina el registro del historial pero no revierte el estado del sistema, de momento no sé implementar que deshaga una acción interiormente, no solo en un registro.

ADR-006 — Excepciones checked personalizadas

Contexto

El sistema tiene reglas que pueden violarse durante su uso normal: IDs duplicados, roles ya ocupados, presupuesto excedido, jugadores sancionados, partidos repetidos.

Decisión

Se crean seis excepciones que extienden Exception (checked): PersonaDuplicadaException, EquipoDuplicadoException, RolNoDisponibleException, PresupuestoExcedidoException, JugadorSancionadoException y PartidoInvalidoException.

Justificación

Se podría haber optado por un booleano pero, necesitaba algo que obligase a comprobar valores, estas excepciones lo hacen y dejan en claro qué está sucediendo, son casos previstos.

ADR-007 — Liga como gestor

Contexto

El sistema necesita un gestor que coordine todas las entidades y mantenga la coherencia

Decisión

Liga actúa como capa de gestión que coordina entidades pero delega las operaciones específicas en ellas. La regla aplicada es: Liga decide qué hacer y cuándo; las entidades deciden cómo. Main solo hace llamadas a métodos de Liga, nunca manipula directamente las estructuras internas.

Justificación

Esta separación en capas (UI → Liga → Entidades) permite cambiar la interfaz sin tocar la lógica. Liga es el único lugar donde se comprueban restricciones que afectan a múltiples entidades, como verificar que un jugador no sea titular ni suplente en ningún equipo antes de eliminarlo.

ADR-008 — Matriz bidimensional Partido[][] para el calendario de jornadas

Contexto

La temporada tiene 14 jornadas con hasta 5 partidos cada una. Se necesita mostrar partidos por jornada, consultar una jornada concreta y acceder a un partido por posición. Una lista plana obligaría a filtrar por jornada en cada consulta

Decisión

Se usa Partido[14][5] en Liga. La primera dimensión representa las jornadas y la segunda los partidos dentro de cada jornada. Durante la generación del calendario, cada partido se coloca en su posición y también se encola en la Queue. consultarJornada(n) accede directamente por índice

Justificación

La matriz y la Queue son estructuras complementarias que almacenan referencias a los mismos objetos. Modificar un Partido desde cualquiera de las dos estructuras afecta al mismo objeto en memoria.

ADR-009 — HashMap para estadísticas de pick por personaje

Contexto

Cada jugador puede usar múltiples personajes a lo largo de la temporada. Para cada personaje se necesita rastrear partidas jugadas y winrate de forma independiente.

Decisión

Se usa `Map<PersonajeJuego, EstadisticasDelPickPersonaje>` dentro de `Jugador`. `PersonajeJuego` implementa `equals()` y `hashCode()` basados en el nombre del personaje, lo que permite usarlo como clave. Al registrar un resultado se comprueba si el personaje ya tiene entrada; si no, se crea una nueva.

Justificación

Sin implementar `equals()` y `hashCode()`, dos objetos con el mismo nombre de personaje serían tratados como claves distintas, duplicando entradas.

ADR-010 — Comparable<Jugador> para ordenación por rendimiento

Contexto

La clasificación de jugadores por rendimiento se necesita en varios puntos del sistema. Sin `Comparable`, cada ordenación requeriría un `Comparator` inline, duplicando lógica y acoplando el criterio de orden a cada llamada.

Decisión

`Jugador` implementa `Comparable<Jugador>`. El criterio es `calcularRendimiento()` en orden descendente (mayor rendimiento primero), `calcularRendimiento()` devuelve la media de `nivelMecanico` y `nivelEstrategico`.

Justificación

Con `Comparable` implementado, cualquier `List<Jugador>` se ordena sin argumentos adicionales. El rendimiento puede cambiar durante la temporada, por lo que el orden se recalcula en cada llamada a `sort()`.

ADR-011 — HashSet temporal para validar roles duplicados en convocatoria

Contexto

Antes de disputar un partido, hay que verificar que la convocatoria no tenga roles repetidos.

Decisión

Se crea un `HashSet<Rol>` local al método `convocatoriaValida()`. El set existe solo durante la validación y se descarta al terminar. `HashSet.add()` devuelve `false` si el elemento ya existía.

Justificación

`HashSet` es efímero, por lo que se crea, se usa y se destruye en cada llamada al método.

ADR-012 — Interfaz Entrenable separada de la jerarquía de herencia

Contexto

Los jugadores pueden mejorar entrenando. Colocarla directamente en Jugador funcionaría pero fuerza a que otras clases futuras no puedan usarla.

Decisión

Se define la interfaz Entrenable con los métodos entrenar() y calcularRendimiento(). Jugador la implementa. Entrenador no, porque un coach no entrena sus habilidades del mismo modo que un jugador.

Justificación

Esta distinción permite en el futuro añadir nuevas entidades entrenables sin tocar la jerarquía de herencia, simplemente implementando la interfaz.

ADR-013 — IncidenciaLog

Contexto

Las incidencias pueden afectar a un jugador, un equipo, un entrenador o ser de tipo técnico genérico sin afectado concreto.

Decisión

IncidenciaLog contiene referencias opcionales (jugador, equipo, entrenador) que pueden ser null según el tipo de incidencia. Los métodos de consulta comprueban si son null antes de acceder.

Justificación

Con este diseño, una sola clase cubre todos los casos posibles, si la incidencia afecta a un jugador se rellena ese campo, si afecta a un equipo se rellena el otro, y los que no apliquen simplemente se dejan vacío

ADR-014 — Disputa de partido en dos fases: consulta (peek) y ejecución (poll)

Contexto

Al disputar un partido, el sistema necesita mostrar la información antes de ejecutarlo, pedir datos al usuario (puntuación, MVP) y solo entonces registrar el resultado.

Decisión

La funcionalidad se divide en dos métodos en Liga. Fase 1: disputarSiguientePartido() devuelve el partido con peek() sin extraerlo. Fase 2: registrarDisputaPartido() valida convocatorias, registra el resultado y solo entonces extrae con poll() y mueve al historial.

Justificación

La separación garantiza que si la validación de la convocatoria lanza una excepción, el partido permanece en la cola y puede intentarse de nuevo una vez resuelta la incidencia

ADR-015 — Verificación antes de eliminar una persona

Contexto

Cuando se elimina una persona, puede seguir siendo referenciada por otros objetos: un jugador puede ser titular o suplente, un entrenador puede estar asignado a un equipo. Eliminar sin comprobar dejaría referencias estancadas causando `NullPointerException`

Decisión

`Liga.eliminarPersona()` recorre todos los equipos y verifica tres condiciones: si es el entrenador del equipo, si es titular en algún equipo, si es suplente en algún equipo. Si alguna se cumple, devuelve `false` con un mensaje.

Justificación

El método devuelve `boolean` en lugar de lanzar excepción porque no es un error del sistema sino una restricción que he impuesto

ADR-016 — `calcularCosteMensual()` abstracto con fórmulas distintas por subclase

Contexto

El coste mensual depende del rendimiento y los méritos, pero los criterios son radicalmente distintos entre jugadores (MVPs) y entrenadores (años de experiencia), haciendo imposible una fórmula única en la clase padre.

Decisión

`calcularCosteMensual()` se declara abstract en `PersonaLiga`. `Jugador` implementa `salarioBase + (mvpTotales * 10)`. `Entrenador` implementa `salarioBase + (experiencia * 50)`. `Equipo.calcularCostePlantilla()` recorre la plantilla llamando al método polimórficamente sin ningún `instanceof`.

Justificación

El polimorfismo hace que no haga falta un `instanceof`.

ADR-017 — Actualización de estadísticas en Partido

Contexto

Al registrar el resultado de un partido hay que actualizar victorias/derrotas de equipos, puntos a favor/en contra y estadísticas individuales de jugadores. Por lo que no puede estar ni en el `Main` ni en la clase gestora

Decisión

`Partido.registrarResultado()` es el único punto de entrada público. Internamente delega en dos métodos privados: `actualizarEstadisticasEquipos()` y `actualizarEstadisticasJugadores()`

Justificación

Si `registrarResultado()` lanza excepción, ninguna estadística se modifica.

ADR-018 — validarPresupuesto() privado reutilizado por todos los fichajes

Contexto

Antes de fichar a cualquier persona (titular, suplente o entrenador) hay que verificar que el coste acumulado no supere el presupuesto y se debe evitar duplicar código

Decisión

Se extrae un método privado validarPresupuesto(PersonaLiga persona) en Equipo que calcula calcularCostePlantilla() + calcularCosteMensual() de la persona nueva y lanza PresupuestoExcedidoException si supera el límite. Los tres métodos de fichaje lo invocan

Justificación

El parámetro de tipo PersonaLiga permite reutilizar el mismo método para jugadores y entrenadores, aprovechando el polimorfismo de calcularCosteMensual()

ADR-019 — identificador declarado final en PersonaLiga

Contexto

El identificador actúa como clave en el HashSet de Liga. El identificador es el dato de identidad de una persona y no debería cambiar.

Decisión

El atributo identificador en PersonaLiga se declara final. Solo existe getter, no setter. El valor se establece una única vez en el constructor y no puede cambiar durante toda la vida del objeto. Los demás atributos sí tienen setters porque pueden cambiar legítimamente.

Justificación

Garantiza la integridad de las estructuras de búsqueda. Se trata como si fuese un DNI, no puede cambiar nunca y es único de cada persona

4. Problemas encontrados y cómo se han resuelto

4.1. Consistencia entre estructuras de datos redundantes

La necesidad de mantener sincronizados los HashSets auxiliares con las listas principales de Liga fue duro de narices. Si en algún punto del código se añadía un elemento a la lista pero no al set, o viceversa, el sistema permitiría duplicados.

La solución fue centralizar todas las operaciones de alta y baja en Liga, asegurando que cada método que modifica una lista principal también modifica su HashSet correspondiente. Nunca se añade o elimina directamente desde Main o desde las entidades, siempre a través de los métodos de Liga.

4.2. Disputar un partido sin perderlo de la cola en caso de error

El primer diseño intentaba hacer peek(), pedir datos al usuario y luego poll() en un único método. El problema era que si el usuario introducía datos inválidos, el partido ya había sido extraído de la cola y se perdía de partidos pendientes.

La solución fue separar la operación en dos métodos distintos: disputarSiguientePartido() que usa peek() y devuelve el partido sin extraerlo, y registrarDisputaPartido() que valida, registra y solo entonces hace poll(). Si la validación lanza excepción en la segunda fase, el partido permanece en la cola y puede intentarse de nuevo.

4.3. Evitar referencias estancadas al eliminar personas

Al implementar la baja de personas, no se había contemplado inicialmente que un jugador podría estar asignado como titular o suplente en un equipo, o un entrenador podría estar dirigiendo uno. Eliminar la persona del ArrayList de Liga sin comprobación dejaba referencias por ahí en los arrays y listas de Equipo.

La solución fue implementar la verificación de integridad en la clase Liga eliminarPersona() antes de eliminar, recorriendo todos los equipos y comprobando las tres posibles dependencias (titular, suplente, entrenador). El método devuelve false con un mensaje si no se puede eliminar, en lugar de lanzar excepción.

4.4. PersonajeJuego como clave de HashMap sin equals/hashCode

Al implementar el HashMap de estadísticas por personaje, los primeros casos de uso mostraba que el mismo personaje creaba entradas duplicadas en el map. El problema era que PersonajeJuego no sobrescribía equals() ni hashCode(), se usaba la dirección en memoria como criterio de igualdad.

La solución fue implementar equals() y hashCode() en PersonajeJuego basándolos únicamente en el nombre del personaje. Con esto, dos objetos PersonajeJuego distintos con el mismo nombre se consideran la misma clave, evitando entradas duplicadas en el HashMap.

4.5. Recorrer la pila de historial sin vaciarla

El método `mostrarHistorial()` necesitaba recorrer todos los elementos de la pila de más reciente a más antiguo, pero el único modo de acceder a todos los elementos de un `Stack` es iterando o haciendo `pop()`. Iterar directamente habría mostrado los elementos de abajo hacia arriba y hacer `pop()` habría vaciado la pila original.

La solución fue crear una copia temporal de la pila, añadiendo todos los elementos con `addAll()`, y hacer `pop()` sobre la copia. La pila original permanece intacta y la copia se descarta al salir del método.

4.6. Control del presupuesto en todos los tipos de fichaje

Al añadir la validación de presupuesto, el primer enfoque duplicaba la lógica de cálculo en `ficharTitular()`, `ficharSuplente()` y `setEntrenador()`.

La solución fue extraer el método privado `validarPresupuesto(PersonaLiga persona)` y hacerlo la primera llamada en cada uno de los tres métodos de fichaje. Al recibir un `PersonaLiga` genérico y delegar en `calcularCosteMensual()` polimórfico, el mismo código funciona tanto para jugadores como para entrenadores sin ningún `instanceof`.

5. Excepciones personalizadas

El proyecto incluye seis excepciones personalizadas que extienden Exception (checked).

Excepción	Se lanza en	Regla que protege
PersonaDuplicadaException	Liga.altaPersona()	No puede haber dos personas con el mismo ID
EquipoDuplicadoException	Liga.crearEquipo()	No puede haber dos equipos con el mismo nombre
RolNoDisponibleException	Equipo.ficharTitular(), sustituirJugador()	No puede haber dos titulares con el mismo rol
PresupuestoExcedidoException	Equipo.validarPresupuesto()	El coste total no puede superar el presupuesto del equipo
JugadorSancionadoException	Equipo.convocatoriaValida()	Un jugador sancionado no puede entrar en una convocatoria
PartidoInvalidoException	Partido(), registrarResultado()	Un equipo no puede jugar contra sí mismo ni repetir un partido

Las excepciones se capturan siempre en Main, que es la capa de interfaz y la responsable de mostrar los mensajes. Los métodos intermedios las propagan con throws.

6. Conclusiones

El proyecto me ha reforzado mentalmente y ha supuesto un aprendizaje mayor al de estos últimos meses, he podido entender realmente cómo deben funcionar determinadas clases y funciones. La herencia reduce la duplicación de código, las interfaces permiten añadir capacidades extras, las estructuras de datos como Queue, Stack, HashSet, HashMap son muy eficientes...

La decisión de documentar cada elección de diseño como ADR ha sido gracias a Pablo y resultado muy útil para razonar sobre las alternativas descartadas y poder estructurar mis pensamientos e ideas. Al principio me costó mucho entender la documentación y saber por dónde empezar, el main me ha dado terribles dolores de cabeza para poder comprender cómo trabajaba la clase liga y dónde iba cada método que quería usar pero aun entendiendo que habrá fallos por algún sitio, he quedado bastante satisfecho con mi proyecto, aunque el formateado de texto para tablas y menús ha sido trabajo de la IA y me ha ayudado a encontrar solución a problemas de cohesión entre distintas clases.